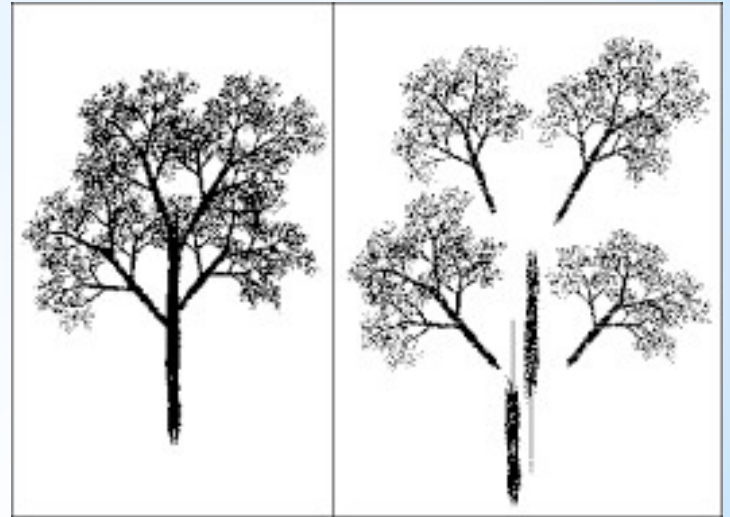
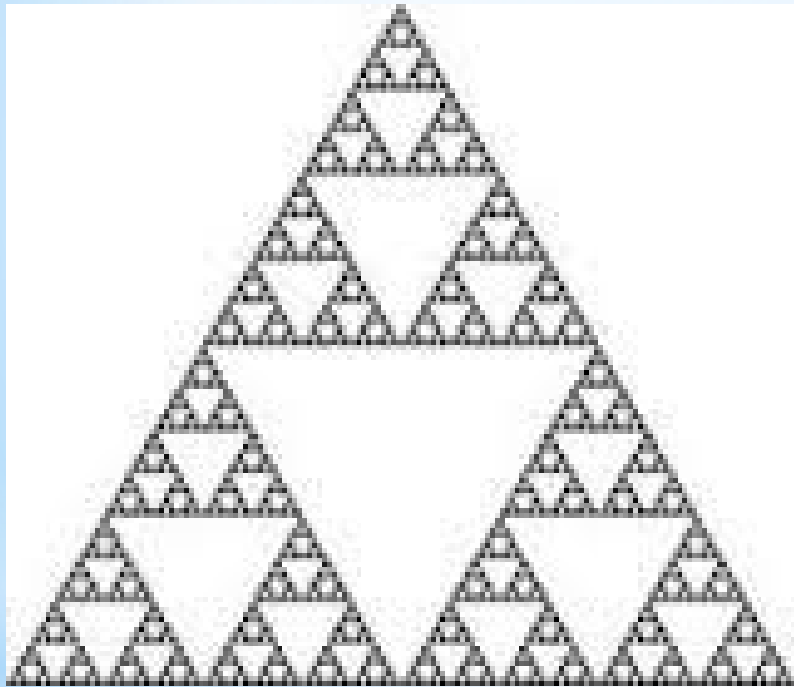


Recursion in C

Concept of Recursion



Recursion

- A process by which a function calls itself repeatedly.
 - Such a function is called a **recursive function**
- Recursion may be direct or cyclically in a chain
 - Direct recursion.
 - When a function $f(\dots)$ calls $f(\dots)$.
 - Cyclically in a chain recursion.
 - $f_1(\dots)$ calls $f_2(\dots)$, $f_2(\dots)$ calls $f_3(\dots)$... $f_i(\dots)$ calls $f_1(\dots)$

Some Examples of Recursion

- Example 3: GCD of two positive integers

$$\text{gcd}(10, 15) = 5, \text{gcd}(11, 13) = 1$$

$$\text{gcd}(m, n) = \text{gcd}(m-n, n), \text{ if } m > n \text{ else } \text{gcd}(m, n-m)$$

- Example 4: Recursion formula

$$T(n) = n + 2 \times T(n-1)$$

$$T(100) = ?$$

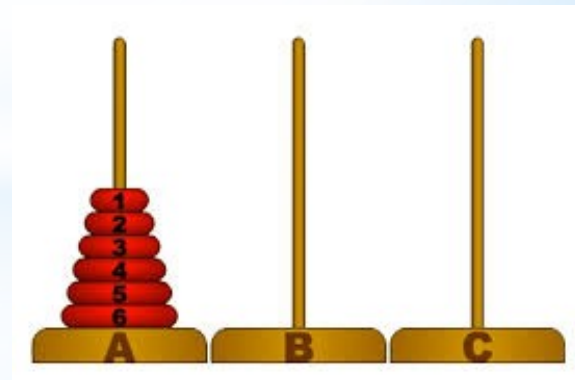
- Example 4: Tower of Hanoi

Move n disks from A C

= move $(n-1)$ disks from A to B

+ move the disk from A to C

+ move $(n-1)$ disk from B to C



Some Examples of Recursion

- Example 1: Factorial calculation

$$n! = n \times (n-1) \times (n-2) \times \dots \times 3 \times 2 \times 1$$

$$n! = n \times (n-1) !$$

$$\text{factorial}(n) = n \times \text{factorial}(n-1)$$

- Example 2: Fibonacci number sequence

1, 1, 2, 3, 5, 8, 13, 21,

$$n_i = n_{i-1} + n_{i-2}$$

$$\text{fibonacci}(n) = \text{fibonacci}(n-1) + \text{fibonacci}(n-2)$$

Important Points to be Noted

- For a function to be written in recursive form, two conditions are to be satisfied:
 - **Condition 1**
 - It should be possible to express the problem in recursive form.
 - **Condition 2**
 - The problem statement must include a stopping condition.

Important Points to be Noted

- It should be possible to express the problem in recursive form.

Examples

$\text{factorial}(n) = n \times \text{factorial}(n-1)$

$\text{fibonacci}(n) = \text{fibonacci}(n-1) + \text{fibonacci}(n-2)$

$\text{gcd}(m, n) = \text{if } m > n \text{ gcd}(m-n, n) \text{ else gcd}(m, n-m)$

$T(n) = n + 2 \times T(n-1)$

Important Points to be Noted

- The problem statement must include a stopping condition.

Examples

$\text{factorial}(n) = n \times \text{factorial}(n-1)$

- $\text{factorial}(0) = 1$

$\text{fibonacci}(n) = \text{fibonacci}(n-1) + \text{fibonacci}(n-2)$

- $\text{fibonacci}(0) = 0; \text{fibonacci}(1) = 1$

$\text{gcd}(m, n) = \text{if } m > n \text{ gcd}(m-n, n) \text{ else gcd}(m, n-m)$

- $\text{gcd}(m, m) = m$

$T(n) = n + 2 \times T(n-1)$

- $T(0) = 0$

Some Examples of Recursion

Recursive Function for $n!$

$\text{fact}(0) = 1$

$\text{fact}(n) = n \times \text{fact}(n-1), \text{ if } n > 0$

```
#include <stdio.h>

int fact(int n)
{
    if (n == 0)
        return 1;
    else
        return (n * fact(n-1));
}

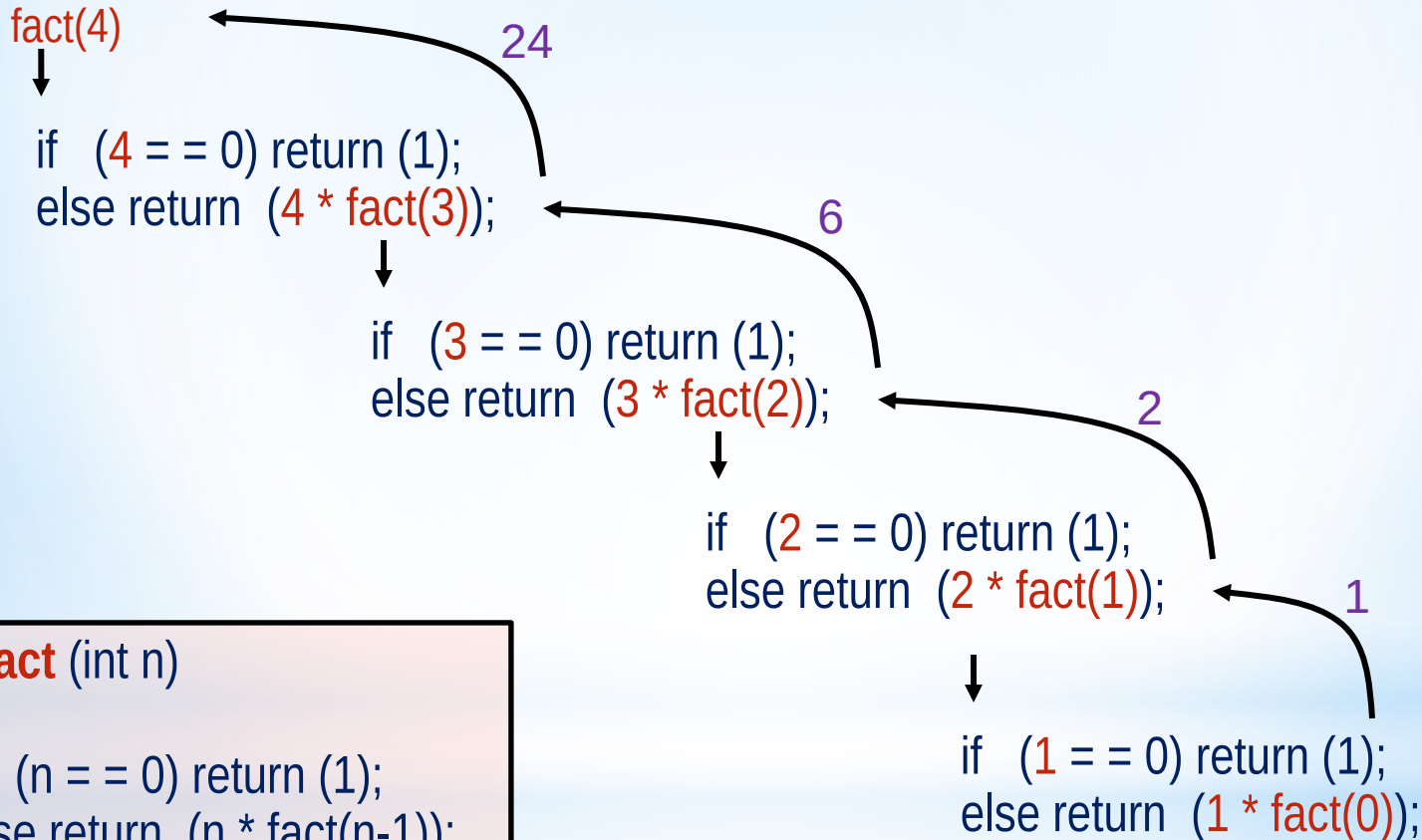
void main()
{
    int x;
    scanf("%d", &x);
    printf ("Factorial of %d is: %d", x, fact(x));
}
```

Factorial Execution

When a recursive program is executed, the recursive function calls are not executed immediately.

- * They are kept aside (on a stack) until the stopping condition is encountered.
- * The function calls are then executed in reverse order.

Factorial Execution



```
int fact (int n)  
{  
    if (n == 0) return (1);  
    else return (n * fact(n-1));  
}
```

Factorial Execution

- **First, the function calls will be processed**

$$\text{fact}(4) = 4 * \text{fact}(3)$$

$$\text{fact}(3) = 3 * \text{fact}(2)$$

$$\text{fact}(2) = 2 * \text{fact}(1)$$

$$\text{fact}(1) = 1 * \text{fact}(0)$$

- **The actual values return in the reverse order**

$$\text{fact}(0) = 1$$

$$\text{fact}(1) = 1 * 1 = 1$$

$$\text{fact}(2) = 2 * 1 = 2$$

$$\text{fact}(3) = 3 * 2 = 6$$

$$\text{fact}(4) = 4 * 6 = 24$$

Recursive Function for $F(n)$

$\text{fibonacci}(0) = 0; \text{fibonacci}(1) = 1$
 $\text{fibonacci}(n) = \text{fibonacci}(n-1) + \text{fibonacci}(n-2), \text{ if } n > 2$

```
#include <stdio.h>

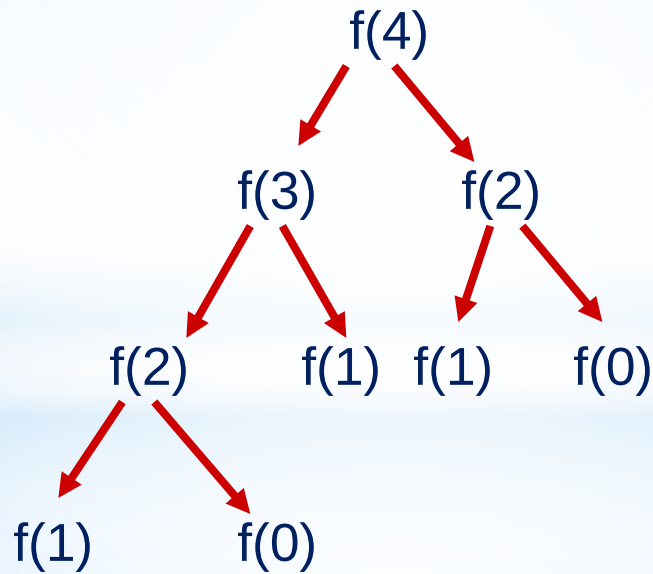
int fibo(int n)
{
    if (n < 2)
        return n;
    else
        return (fibo(n-1) + fibo(n-2));
}

void main()
{
    int x;
    scanf("%d", &x);
    printf ("%d-th Fibonacci number is %d \n", x, fibo(x));
}
```


Fibonacci Series Execution

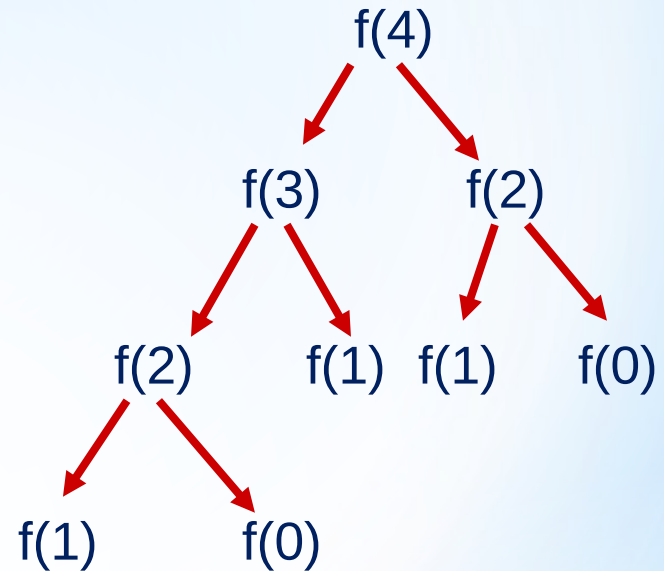
$f(0) = 0$
 $f(1) = 1$
 $f(n) = f(n-1) + f(n-2), \text{ if } n > 1$

```
int f(int n)
{
    if (n < 2)
        return (n);
    else
        return (f(n-1) + f(n-2));
}
```



Tracing Execution

* How many times is the function called when evaluating $f(4)$?



called 9 times

***Inefficiency:**

* Same thing is computed several times.

Recursive Function for $gcd(m,n)$

$gcd(m,m) = m;$
 $gcd(m,n) = gcd(m-n, n) \quad \text{if } m > n$
 $\quad = gcd(m, n-m) \quad \text{else}$

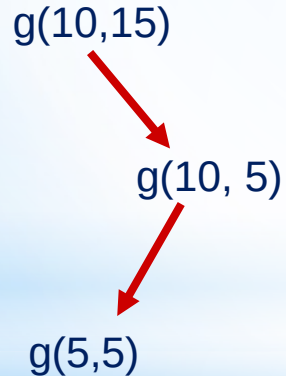
```
#include <stdio.h>

int gcd(int m, int n)
{
    if (m == n)
        return m;
    else
        if (m > n) return (gcd(m-n, n);
        else return gcd(m, n-m);
}

void main()
{
    int x, y;
    scanf("x = %d, y = %d", &x, &y);
    printf ("GCD of %d and %d is %d", x, y, gcd(x, y));
}
```

GCD Execution

$\text{gcd}(m, m) = m;$
 $\text{gcd}(m, n) = \text{gcd}(m-n, n) \quad \text{if } m > n$
 $\quad = \text{gcd}(m, n-m) \quad \text{else}$



```
int gcd(int m, int n)
{
    if (m == n)
        return m;
    else
        if(m > n) return (gcd(m-n, n);
        else return gcd(m, n-m);
}
```

Performance Comparison of GCDs

$\text{gcd}(m,n) = m, \text{ if } m = n$
 $= \text{gcd}(m-n,n) \text{ if } m > n$
 $= \text{gcd}(m,n-m) \text{ else}$

```
int gcd(int m, int n)
{
    if (m == n)
        return m;
    else
        if(m > n) return (gcd(m-n,n);
        else return gcd(m, n-m);
}
```

$\text{gcd}(m,n) = m, \text{ if } (m = n) \text{ or } n = 0;$
 $= \text{gcd}(n,m) \text{ if } n > m$
 $= \text{gcd}(m\%n,m)$

```
int gcd(int m, int n)
{
    if ((m == n) || (n == 0))
        return m;
    else
        if(n > m) return (gcd(n,m);
        else return gcd(m, n%m);
}
```

Compare the performance of the above-mentioned two versions of gcd calculations

Recursive Function for $T(n)$

$$T(0) = 0$$

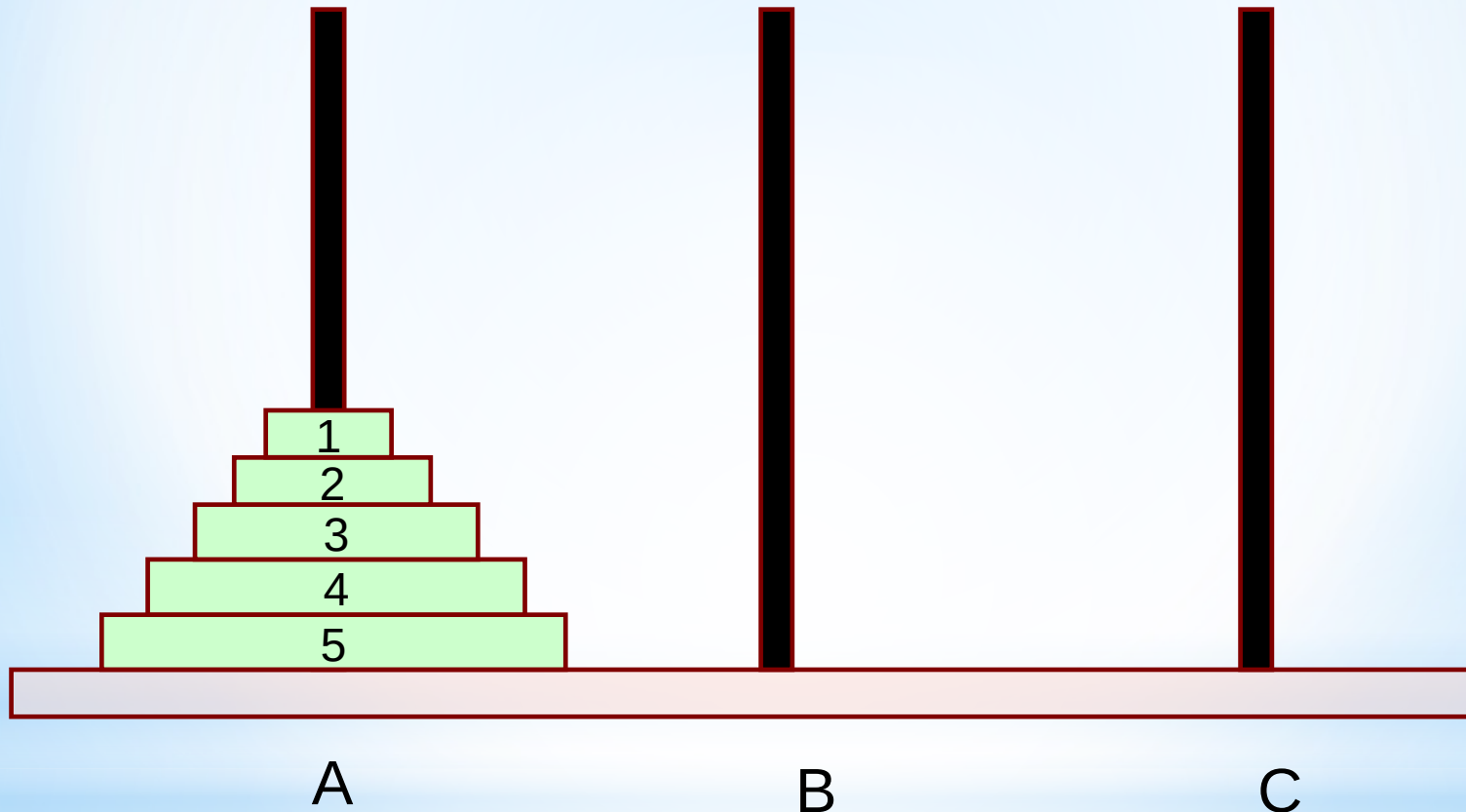
$$T(n) = n + 2 \times T(n-1), \text{ if } n > 0$$

```
#include <stdio.h>

int T(int n)
{
    if (n == 0)
        return 0;
    else
        return (n + 2*T(n-1));
}

void main()
{
    int x;
    scanf("%d", &x);
    printf ("T(%d)= %d", x, T(x));
}
```

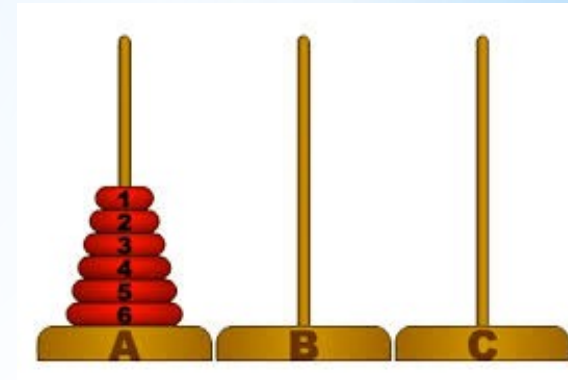
Towers of Hanoi Problem



Towers of Hanoi Problem

- **The problem statement**

- Initially all the disks are stacked on the A pole.
- Required to transfer all the disks to the C pole.
 - Only one disk can be moved at a time.
 - A larger disk cannot be placed on a smaller disk.
- C pole is used for temporary storage of disks.



Towers of Hanoi Problem

- Recursive statement of the general problem of n disks

- **Step 1:**

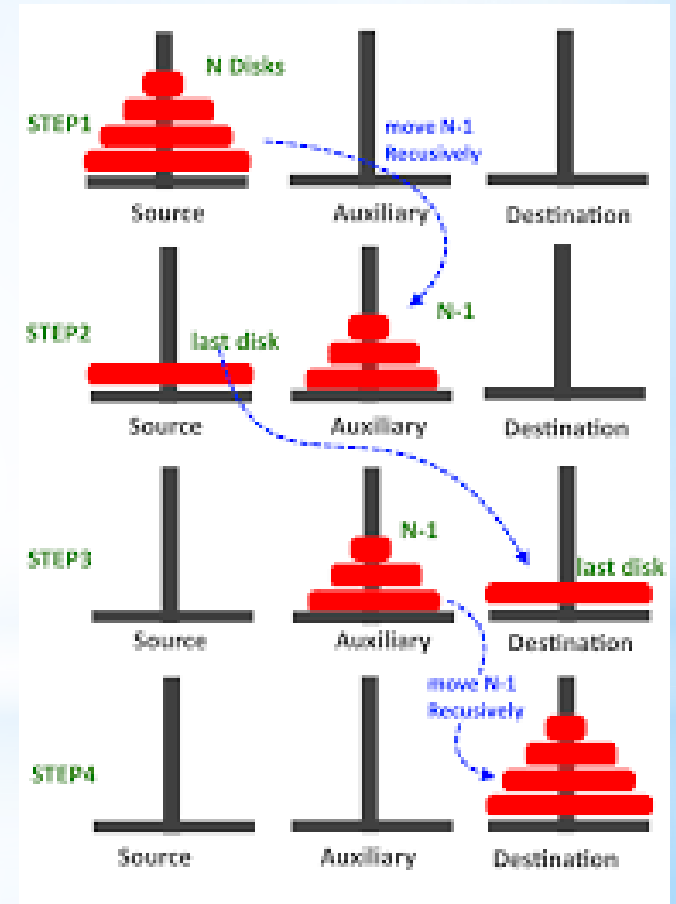
- Move the top $(n-1)$ disks from A to B

- **Step 2:**

- Move the largest disk from A to C.

- **Step 3:**

- Move the $(n-1)$ disks from B to C.



Towers of Hanoi Problem

```
#include <stdio.h>

void move(int n, char A, char B, char C);

int main()
{
    int n;    /* Number of disks */
    scanf ("%d", &n);
    move (n, 'A', 'B', 'C');
    return 0;
}

void move (int n, char A, char B, char C)
{
    if (n > 0) {
        move (n-1, A, C, B);
        printf ("Move disk %d from %c to %c \n", n, A, C);
        move (n-1, B, C, A);
    }

    return;
}
```

Towers of Hanoi – Execution

3

Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C

4

Move disk 1 from A to B
Move disk 2 from A to C
Move disk 1 from B to R
Move disk 3 from A to B
Move disk 1 from C to L
Move disk 2 from C to B
Move disk 1 from A to B
Move disk 4 from A to C
Move disk 1 from B to C
Move disk 2 from B to A
Move disk 1 from C to A
Move disk 3 from B to C
Move disk 1 from A to B
Move disk 2 from A to C
Move disk 1 from B to C

5

Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C
Move disk 4 from A to B
Move disk 1 from C to B
Move disk 2 from C to A
Move disk 1 from B to A
Move disk 3 from C to B
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 5 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C
Move disk 3 from B to A
Move disk 1 from C to B
Move disk 2 from C to A
Move disk 1 from B to A
Move disk 4 from B to C
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C

How many moves are
required for n disks?

Points to be Noted

1. Every recursive program can also be written without recursion.
2. Recursion is used for programming convenience, not for performance enhancement.
3. Sometimes, if the function being computed has a nice recurrence form, then a recursive code may be more readable.

Performance warning!

Speed: Avoid Fibonacci-style recursive programs, which result in an exponential “explosion” of calls.

Memory: Recursive calls take consume additional memory.

Recursion Versus Iteration

- **Repetition**

- Iteration: explicit loop
- Recursion: repeated function calls

- **Termination**

- Iteration: Loop condition fails
- Recursion: Base case recognized

- **Both can have infinite loops**

- Be careful about the termination condition

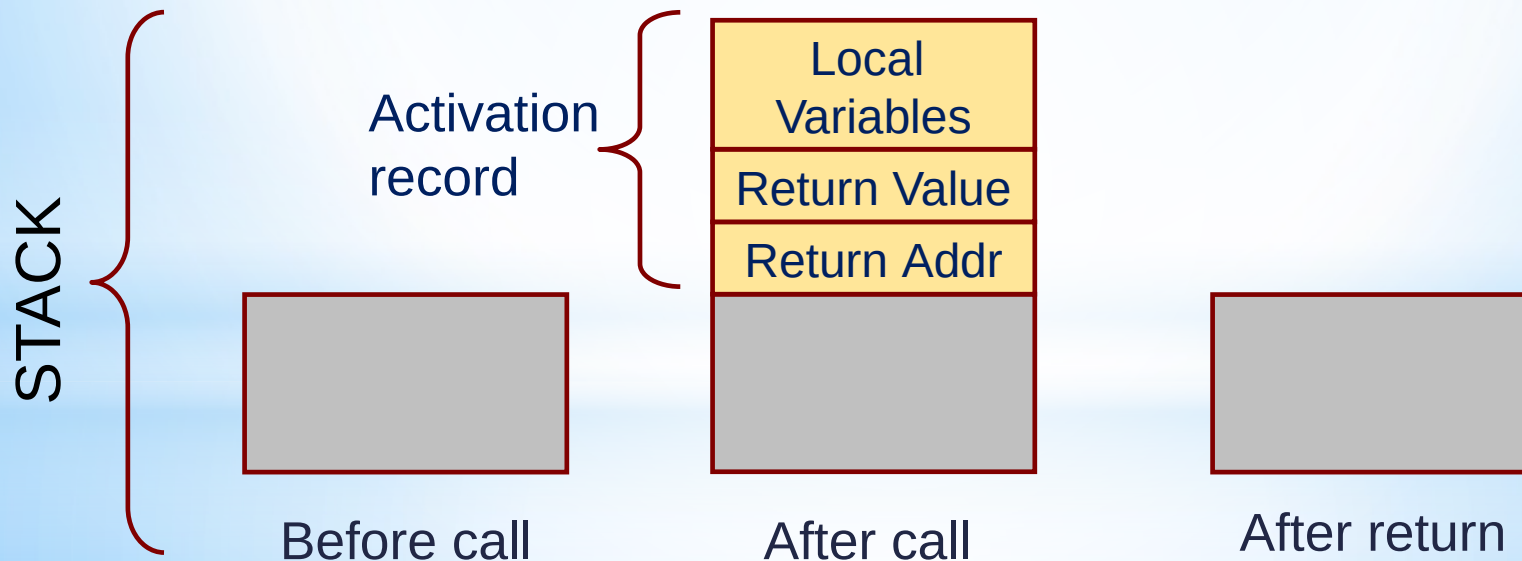
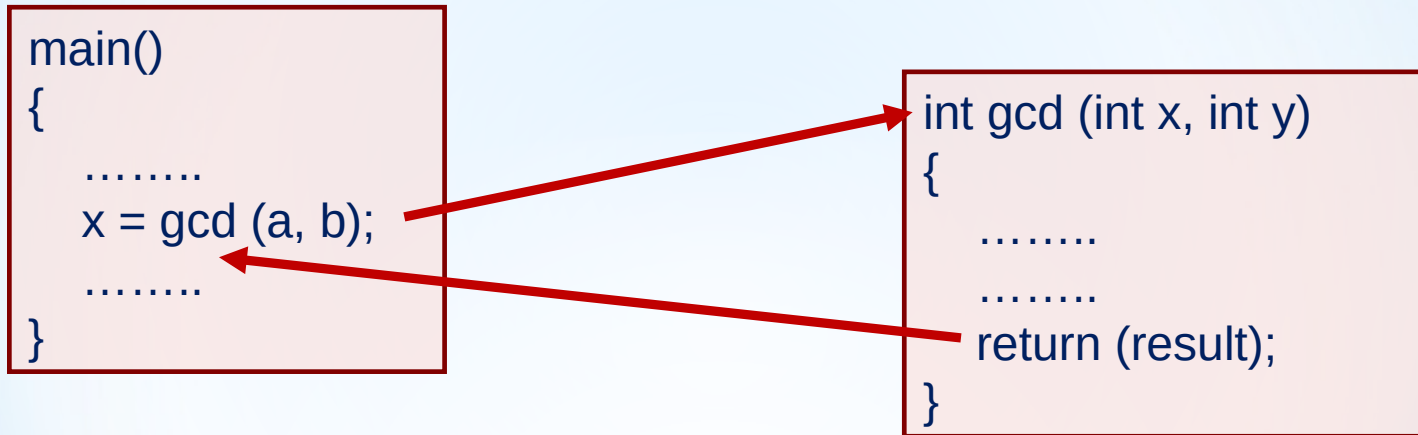
- **Balance**

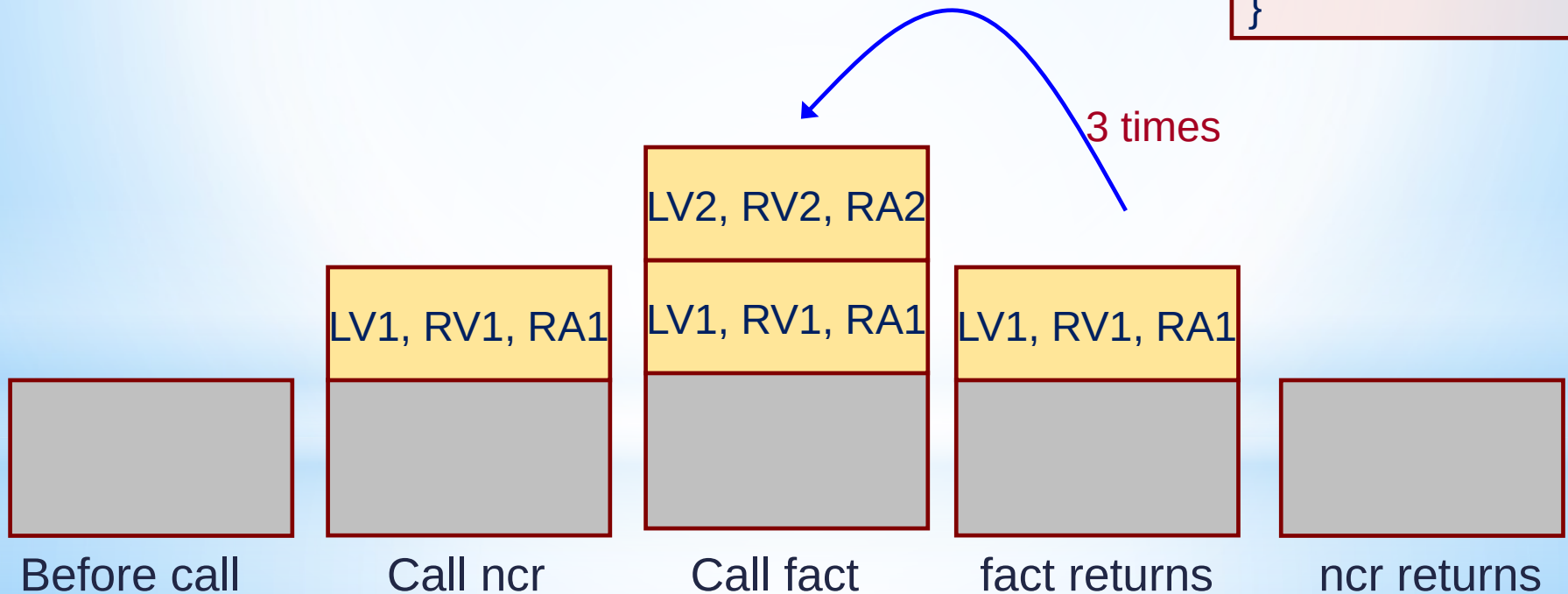
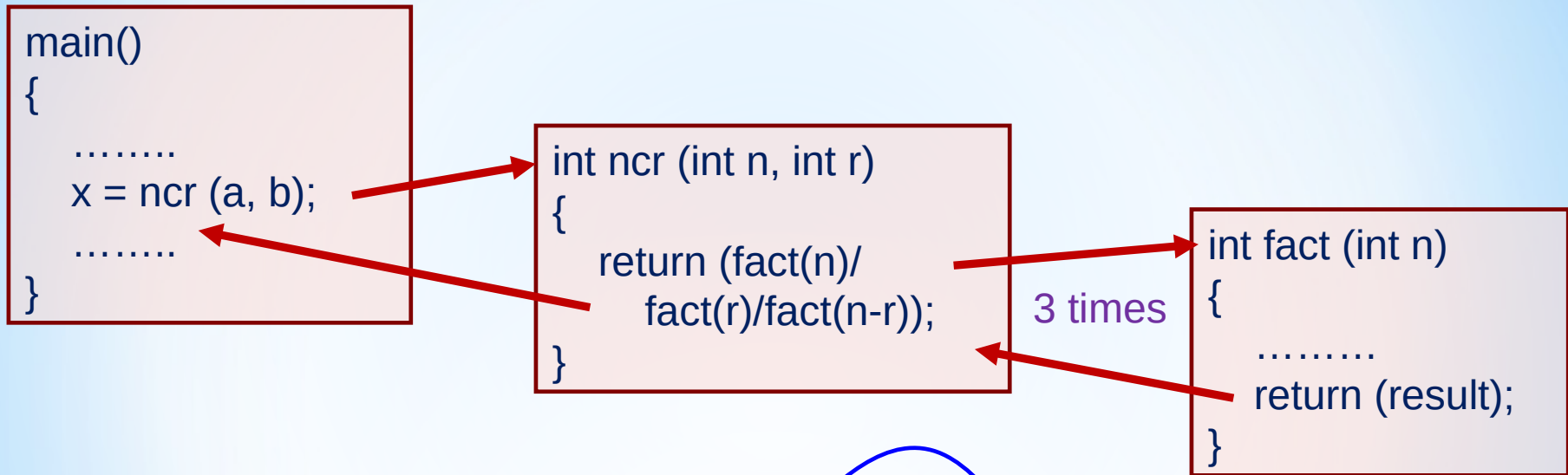
- Choice between performance (iteration) and good software engineering (recursion).

How are function calls implemented?

- The following applies in general, with minor variations that are implementation dependent.
 - The system maintains a **stack** in memory.
 - Stack is a **last-in first-out** structure.
 - Two operations on stack, **push** and **pop**.
 - Whenever there is a function call, the **activation record** gets **pushed** into the stack.
 - Activation record consists of the **return address** in the calling program, the **return value** from the function, and the **local variables** inside the function.

Function calls implementation





What happens for recursive calls?

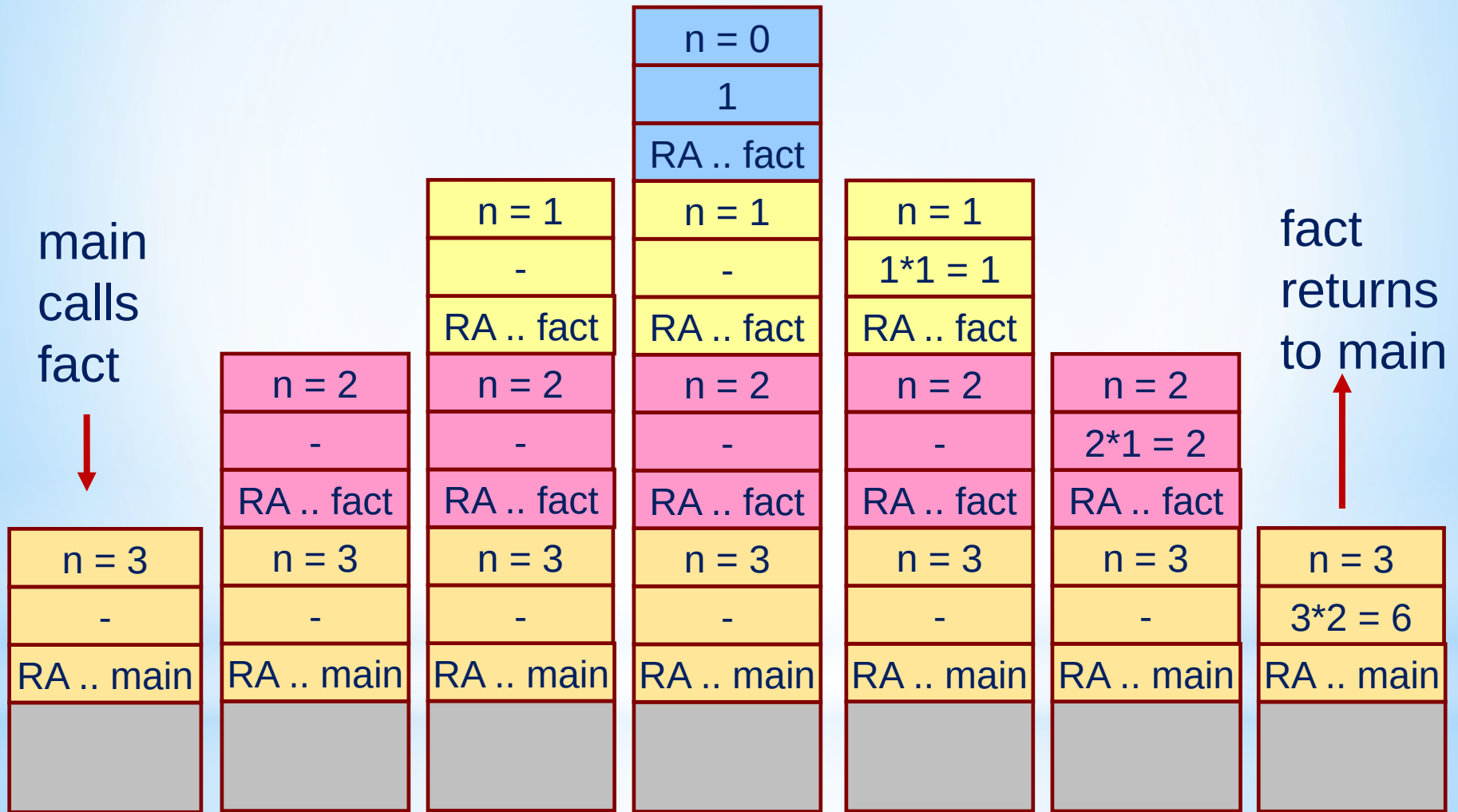
- **What we have seen**
 - Activation record gets **pushed** into the stack when a **function call** is made.
 - Activation record is **popped** off the stack when the **function returns**.
- **In recursion, a function calls itself.**
 - Several function calls going on, with none of the function calls returning back.
 - Activation records are **pushed** onto the stack continuously.
 - **Large stack space** required.
 - Activation records keep **popping** off, when the **termination condition** of recursion is reached.

Example:: main() calls fact(3)

```
main()
{
    int  n;
    n = 3;
    printf ("%d \n", fact(n) );
}
```

```
int  fact (n)
int  n;
{
    if    (n == 0)
        return (1);
    else
        return(n * fact(n-1));
}
```

TRACE OF THE STACK DURING EXECUTION





Trace the activation records for the following version of Fibonacci sequence.

```
#include <stdio.h>
int f(int n)
{
    int a, b;
    if (n < 2) return (n);
    else {
        a = f(n-1);
        b = f(n-2);
        return (a+b); }
}

main() {
    printf("Fib(4) is: %d \n", f(4));
}
```

X

Y

main

Local
Variables
(n, a, b)

Return Value

Return Addr
(either **main**,
or **X**, or **Y**)